
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 12

Estruturas de Dados em C

Exercícios de Autorrevisão (12.1–12.5)

Resolvidos passo a passo, abordagem incremental
Capítulos 1 a 12

Construindo sobre os Capítulos Anteriores

No Capítulo 12 chegamos ao auge da progressão Deitel: **estruturas de dados dinâmicas**. Tudo o que aprendemos até aqui converge:

- **Estruturas autorreferenciadas** – `struct` contendo ponteiro para o próprio tipo (Cap. 10 + Cap. 7)
- **Alocação dinâmica** – `malloc`, `calloc`, `realloc`, `free` de `<stdlib.h>`
- **Listas encadeadas** – coleções dinâmicas que crescem em tempo de execução
- **Pilhas** (LIFO) – inserir/remover sempre no topo
- **Filas** (FIFO) – inserir na cauda, remover da cabeça
- **Árvores binárias** – estrutura não-linear com dois links por nó
- **Travessias** – in-order, pré-ordem, pós-ordem, nível

Boa prática de programação

As estruturas de dados deste capítulo são fundação para tudo: bancos de dados, compiladores, sistemas operacionais, gerenciadores de pacotes, sistemas de arquivos. Toda linguagem moderna (Java, Python, C++) implementa essas mesmas estruturas internamente. Quem aprende em C, aprende como elas funcionam por dentro.

Contents

1	Exercício 12.1 – Preenchimento de Lacunas (15 itens)	2
2	Exercício 12.2 – Lista Encadeada vs. Pilha	3
3	Exercício 12.3 – Pilha vs. Fila	3
4	Exercício 12.4 – Manipulando GradeNode	4
5	Exercício 12.5 – Travessias da Árvore Binária	6

1. Exercício 12.1 – Preenchimento de Lacunas (15 itens)

Resposta detalhada

- a) Uma estrutura **autorreferenciada** forma estruturas de dados dinâmicas.
- b) A função **malloc** aloca memória dinamicamente.
- c) Uma **pilha** é especialização de lista onde nós são inseridos/excluídos apenas do início (topo).
- d) Funções que examinam uma lista sem modificá-la são chamadas de **predicados**.
- e) Uma fila é estrutura de dados **FIFO** (*first-in, first-out*).
- f) O ponteiro para o próximo nó é chamado de **link**.
- g) A função **free** libera memória alocada dinamicamente.
- h) Uma **fila** permite inserções no fim e exclusões no início.
- i) Uma **árvore** é estrutura não-linear, com nós tendo dois ou mais links.
- j) Uma pilha é estrutura **LIFO** (*last-in, first-out*).
- k) Nós de uma árvore **binária** contêm dois membros de link.
- l) O primeiro nó de uma árvore é o nó **raiz**.
- m) Cada link em um nó de árvore aponta para um **filho** ou **subárvore**.
- n) Um nó sem filhos é chamado nó **folha**.
- o) Os três algoritmos de travessia são **in-order** (na ordem), **pré-ordem** e **pós-ordem**.

Resumo das estruturas:

Estrutura	Padrão	Operações principais
Lista encadeada	livre	inserir/remover em qualquer lugar
Pilha	LIFO	push (topo) / pop (topo)
Fila	FIFO	enqueue (cauda) / dequeue (cabeça)
Árvore binária	não-linear	inserir / buscar / travessias

2. Exercício 12.2 – Lista Encadeada vs. Pilha

Resposta detalhada

Diferença principal: flexibilidade de acesso.

Lista encadeada:

- Inserção em *qualquer* posição: início, meio, fim.
- Remoção em *qualquer* posição.
- Suporta percurso e busca em qualquer direção (em listas duplamente encadeadas).

Pilha:

- Inserção (*push*) **apenas no topo**.
- Remoção (*pop*) **apenas do topo**.
- Acesso restrito ao último elemento inserido.

Por que limitar o acesso? Disciplina LIFO é exatamente o que se precisa em vários contextos:

- **Chamadas de função:** parâmetros e variáveis locais são empilhados.
- **Avaliação de expressões:** conversão infix → pós-fix.
- **Desfazer (undo):** última ação → primeira a desfazer.
- **Travessia em profundidade (DFS):** pilha implícita ou explícita.

Uma pilha pode ser *implementada* usando uma lista encadeada (limitando os pontos de acesso), mas conceitualmente é uma estrutura mais restrita.

3. Exercício 12.3 – Pilha vs. Fila

Resposta detalhada

Pilha (LIFO):

- **Um único ponteiro:** para o *topo*.
- Push e pop acontecem no mesmo lugar.
- Último inserido é o primeiro removido.

Fila (FIFO):

- **Dois ponteiros:** para a *cabeça* (front) e para a *cauda* (tail).
- Enqueue ocorre na cauda; dequeue na cabeça.
- Primeiro inserido é o primeiro removido – ordem natural “chegou primeiro, atendido primeiro”.

Aplicações de fila:

- Escalonamento de processos pelo SO (filas de prontos)
- Buffers de E/S (impressora, teclado, rede)
- Travessia em largura (BFS) de grafos
- Simulações de filas de espera (banco, supermercado)

Analogia visual:

- Pilha: como uma pilha de pratos – adiciona/retira só do topo.
- Fila: como uma fila de pessoas – entra atrás, sai na frente.

4. Exercício 12.4 – Manipulando GradeNode

Resposta detalhada

Considere:

```
1 struct gradeNode {
2     char lastName[ 20 ];
3     double grade;
4     struct gradeNode *nextPtr;
5 };
6 typedef struct gradeNode GradeNode;
7 typedef GradeNode *GradeNodePtr;
```

(a) Criar ponteiro inicial (lista vazia):

```
1 GradeNodePtr startPtr = NULL;
```

A convenção NULL indica “lista vazia” – nenhum nó alocado ainda.

(b) Criar novo nó com “Jones”, 91.5:

```
1 GradeNodePtr newPtr;
2 newPtr = malloc( sizeof( GradeNode ) );
3 if ( newPtr != NULL ) {
4     strcpy( newPtr->lastName, "Jones" );
5     newPtr->grade = 91.5;
6     newPtr->nextPtr = NULL;
7 }
```

Sequência clássica: (1) alocar memória, (2) verificar sucesso, (3) preencher dados, (4) inicializar ponteiro next.

(c) Inserir mantendo ordem alfabética:

Inicialmente a lista tem: Jones → Smith.

Inserir “Adams” 85.0: “Adams” precede “Jones”. Como começa no início da lista, previousPtr = NULL e currentPtr aponta para o nó “Jones” (primeiro elemento).

```
1 newPtr->nextPtr = currentPtr;    /* newPtr -> Jones */
2 startPtr = newPtr;              /* head -> Adams */
```

Inserir “Thompson” 73.5: “Thompson” vem depois de “Smith”. Após percorrer a lista, previousPtr aponta para “Smith” (último) e currentPtr = NULL.

```
1 newPtr->nextPtr = currentPtr;    /* Thompson -> NULL
   */
2 previousPtr->nextPtr = newPtr;    /* Smith -> Thompson
   */
```

Inserir “Pritchard” 66.5: “Pritchard” fica entre “Jones” e “Smith”. Após busca, previousPtr aponta para “Jones” e currentPtr para “Smith”.

```
1 newPtr->nextPtr = currentPtr;    /* Pritchard ->
   Smith */
```

```

2  previousPtr->nextPtr = newPtr;      /* Jones ->
    Pritchard */

```

Padrão geral de inserção ordenada:

```

1  /* Encontra posicao de insercao */
2  previousPtr = NULL;
3  currentPtr = startPtr;
4  while ( currentPtr != NULL &&
5         strcmp( newName, currentPtr->lastName ) > 0 )
6  {
7      previousPtr = currentPtr;
8      currentPtr = currentPtr->nextPtr;
9  }
10 /* Insere */
11 if ( previousPtr == NULL ) {           /* inicio */
12     newPtr->nextPtr = startPtr;
13     startPtr = newPtr;
14 } else {                               /* meio ou fim
15     */
16     newPtr->nextPtr = currentPtr;
17     previousPtr->nextPtr = newPtr;
18 }

```

(d) Loop while para imprimir todos os nós:

```

1  currentPtr = startPtr;
2  while ( currentPtr != NULL ) {
3      printf( "Lastname = %s\nGrade = %6.2f\n",
4              currentPtr->lastName, currentPtr->grade );
5      currentPtr = currentPtr->nextPtr;
6  }

```

Este é o **padrão de travessia** de lista: começar no início, avançar pelo nextPtr até NULL.

(e) Loop para liberar todos os nós:

```

1  currentPtr = startPtr;
2  while ( currentPtr != NULL ) {
3      tempPtr = currentPtr;
4      currentPtr = currentPtr->nextPtr;  /* avanca ANTES
5      de liberar */
6      free( tempPtr );
7  }
8  startPtr = NULL;

```

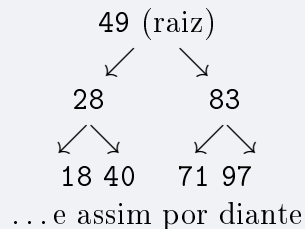
Erro comum de programação

Erro clássico: `free(currentPtr); currentPtr = currentPtr->nextPtr;` – após free, o conteúdo apontado é inválido! Sempre salve o nextPtr *antes* de liberar.

5. Exercício 12.5 – Travessias da Árvore Binária

Resposta detalhada

Árvore dada (15 nós):



A árvore completa, do livro:

- Raiz: 49
- Subárvore esquerda da raiz: 28 (com filhos 18 e 40)
 - 18: filhos 11 e 19
 - 40: filhos 32 e 44
- Subárvore direita da raiz: 83 (com filhos 71 e 97)
 - 71: filhos 69 e 72
 - 97: filhos 92 e 99

Travessia in-order (esquerda, nó, direita) – ordem crescente:

```

1  11  18  19  28  32  40  44  49  69  71  72  83  92  97
    99
  
```

Esta é a propriedade **mágica** de árvores binárias de busca: a travessia in-order produz os elementos em ordem crescente – por isso são usadas em buscas, indexação e ordenação.

Travessia pré-ordem (nó, esquerda, direita):

```

1  49  28  18  11  19  40  32  44  83  71  69  72  97  92
    99
  
```

Cada nó é visitado *antes* de seus filhos. Útil para clonar uma árvore ou serializar sua estrutura.

Travessia pós-ordem (esquerda, direita, nó):

```

1  11  19  18  32  44  40  28  69  72  71  92  99  97  83
    49
  
```

Cada nó é visitado *depois* de seus filhos. Útil para liberar memória (desalocar filhos antes do pai) ou avaliar expressões pós-fixadas.

Algoritmos recursivos (essência):

```

1  void inOrder( const TreeNode *p ) {
2      if ( p == NULL ) return;
3      inOrder( p->left );
4      printf( "%d ", p->data );
5      inOrder( p->right );
6  }
7
8  void preOrder( const TreeNode *p ) {
  
```

```
9      if ( p == NULL ) return;
10     printf( "%d ", p->data );
11     preOrder( p->left );
12     preOrder( p->right );
13 }
14
15 void postOrder( const TreeNode *p ) {
16     if ( p == NULL ) return;
17     postOrder( p->left );
18     postOrder( p->right );
19     printf( "%d ", p->data );
20 }
```

A única diferença entre as três é a ordem das três linhas internas. O nó é processado em uma de três posições – antes, entre ou depois dos filhos.

Boa prática de programação

Como decorar? O nome diz o lugar do *nó*:

- **Pré-ordem**: nó **antes** (pré) dos filhos
- **In-ordem**: nó **entre** (in) os filhos
- **Pós-ordem**: nó **depois** (pós) dos filhos

Gabarito Rápido – Capítulo 12: Autorrevisão

Respostas Resumidas

12.1:

- a) autorreferenciada b) `malloc` c) pilha d) predicados
- e) FIFO f) link g) `free` h) fila i) árvore
- j) LIFO k) binária l) raiz m) filho, subárvore
- n) folha o) in-order, pré-ordem, pós-ordem

12.2: Listas permitem inserção/remoção em qualquer ponto; pilhas restritas ao topo.

12.3: Pilha tem 1 ponteiro (topo); fila tem 2 (cabeça e cauda).

12.4: Padrões clássicos:

- a) `startPtr = NULL`
- b) `malloc`, verificar, preencher, `nextPtr = NULL`
- c) Inserção ordenada: `previous` e `current` rastreiam posição
- d) Travessia: `while (cur != NULL) ... cur = cur->next`
- e) Liberação: salvar `next` antes de `free`

12.5 – Travessias:

- **In-order:** 11 18 19 28 32 40 44 49 69 71 72 83 92 97 99 (ordenado!)
- **Pré-ordem:** 49 28 18 11 19 40 32 44 83 71 69 72 97 92 99
- **Pós-ordem:** 11 19 18 32 44 40 28 69 72 71 92 99 97 83 49